Contender 14: Self-Height Operator Power over Dynamic-Kind System Fω

A constructive large-number proposal for BusyLean

Research package for the BusyLean project

October 2026

Abstract

The current BusyLean incumbent, Contender 13, takes one bounded reflection step through a strongly normalizing, dynamic-kind extension of System Fω. We propose a successor that changes the *amplifier*, not the underlying normalization proof. First, the unary Contender 13 reflection is lifted to countable oracle families while retaining explicit domination of every visible family coordinate. Second, this progressive transformer is closed under BusyLean's operator-stage construction along a schedule obtained by placing another autonomous critical-point closure above Contender 12's schedule. Third, the complete operator stage is iterated a self-fed finite number of times: at input n, the height is the previous full family value $G(n)+1$, where $G(n) = F_{13}(F_{13}(n))$. Finally,

$$F_{14}(n) = P_{G(n)+1}(\underline{G})(G(n), G(n)), \qquad C_{14} = F_{14}(F_{14}(42)).$$

Every operation is transparent, structural, and executable in principle. The supplied Lean development proves the pointwise margin $G(n) < F_{14}(n)$, the required comparison $C_{13} < C_{14}$, and domination of every accepted Fd-program at the construction's diagonal budget. The article separates these formal claims from heuristic proof-theoretic calibration and records the exact validation status of the package.

Main proposed theorem.
`BusyLean.contender_13_lt_contender_14 : contender_13 < contender_14`. The proof is reduced to explicit occurrence inequalities in already established reflection machinery; it does not compare or evaluate either gigantic numeral.

Contents

1 Repository snapshot and design target **3**
1.1 The incumbent in one equation 3

2 Rejected shortcuts **3**

3 Available infrastructure **4**
3.1 Oracle families and their unary code 4
3.2 Progressive transformers 4
3.3 Why oracle monotonicity is deliberately absent 5

4 The new family-aware Fd reflection **5**

5 A stronger autonomous schedule **6**

6 Finite powers of complete operator stages **6**

7 The self-height diagonal **7**
7.1 The strict comparison 7

8 Program-level domination **8**

9 Constructivity and contest compliance **8**

10 Heuristic growth analysis **9**

11 Lean proof engineering **10**
11.1 Never normalize a contender 10
11.2 Use explicit lower-bound occurrences 10
11.3 Separate definition and proof budgets 10

12 Validation status and reproducibility **10**
12.1 Reproduction commands 11

13 Further research **11**

A Complete definition module **11**

B Complete comparison module **13**

C Theorem inventory **16**

1 Repository snapshot and design target

This work targets BusyLean at commit `0cd13c0c2607b4fcb461090a95f356e538c0fc5b`, whose documented incumbent is Contender 13 [BR26]. The repository is a Lean 4 version of the constructive large-number contest initiated in Coq by Cody Roux and inspired by Aaronson's larger-number essay [Rou14, Aar99]. Its relevant acceptance conditions are unusually stringent:

1. the number is a closed, computable `Nat`;
2. the strict comparison with the previous contender is a formal theorem;
3. no `noncomputable`, `partial`, `unsafe`, opaque stand-in, external implementation, `native_decide`, or custom axiom is used;
4. only the repository's permitted logical axioms may occur in the proof closure;
5. the definition may use the previous contender's *construction* as a seed or oracle, but not merely mention the previous closed constant and add one; and
6. the definition and proof must remain within the repository's checking budgets.

The pinned toolchain is Lean 4.34.0. Lean is both the implementation language and the trusted proof checker [DMU21]. Consequently, the central design problem is not to write down a huge informal expression. It is to expose a transparent total program whose strict advantage is witnessed by a small proof term that never asks the elaborator to normalize the number.

1.1 The incumbent in one equation

Write $R_f^{\mathsf{Fd}}(n)$ for the repository function `Fd.reflectFd f n`. It is definitionally

$$R_f^{\mathsf{Fd}}(n) = n + 1 + \max\left(f(n), \max_{t \in \mathcal{T}_{\mathsf{Fd}}(n)} \mathrm{value}_f(n, t)\right), \tag{1}$$

where $\mathcal{T}_{\mathsf{Fd}}(n)$ is the finite list of dynamic-kind terms at budget n. The total evaluator is justified by the repository's strong-normalization proof for the object language. Contender 13 is

$$F_{13}(n) = R^{\mathsf{Fd}}_{\mathrm{family}_{12}}(n), \qquad C_{13} = F_{13}(F_{13}(42)). \tag{2}$$

The language includes a predicative dynamic kind that packages operators of all permitted dyn-free kinds. It is therefore materially stronger than the kind-polymorphic Fω language used by Contenders 10–12. Any credible successor should exploit this stronger language rather than retreat to an older base.

2 Rejected shortcuts

The following constructions would establish a numerical inequality but do not meet the research objective.

Adding one. Defining $C_{14} = C_{13} + 1$ is constructive and trivially larger, but violates the no-laziness rule and contributes no structural growth mechanism.

A bare extra diagonal. Replacing $F_{13}(F_{13}(42))$ by a few more literal nestings of F_{13} is better than addition, yet it still treats the current reflection as a black-box unary function. It leaves unused the repository's generic machinery for families, W-tree schedules, and transformer-level reflection.

A new unnormalized language. Adding impredicative kind universes or unrestricted kind-in-kind would make a larger syntax easy to state but would invalidate the total evaluator unless a new normalization theorem were supplied. In this contest, an unproved termination claim is not a contender.

Busy Beaver or semantic maximization. Maximizing over arbitrary terminating Lean programs, or over machines known to halt only propositionally, would introduce noncomputability or require an oracle. The finite syntactic enumerations in (1) are essential.

The proposal below instead reuses the certified Fd evaluator and makes a qualitative jump in the *reflection architecture*: unary reflection is lifted to family reflection, then to operator reflection, then to a self-height finite tower of complete operator closures.

3 Available infrastructure

3.1 Oracle families and their unary code

BusyLean uses a family

$$s : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$$

as a countable collection of unary oracle functions. The already defined `packFamily` maps such a family to one unary oracle by a computable pairing code. Hence an Fd program can dynamically select both a family coordinate and an argument while the metalanguage still passes a single function $\mathbb{N} \to \mathbb{N}$.

For progress proofs, it is not enough that the object language can in principle query the family. The reflector must contain every relevant seed value as an *explicit* lower bound. BusyLean's `prefixMax` provides

$$\mathrm{prefixMax}(s, n) = \max_{i \le n} s(i, n).$$

The visibility restriction $i \le n$ is exactly the hypothesis used by the generic notion of a progressive transformer.

3.2 Progressive transformers

A transformer maps oracle families to oracle families:

$$T : \mathsf{Family} \to \mathsf{Family}.$$

The existing predicate is

$$\mathsf{Progressive}(T) \ :\Longleftrightarrow\ \forall s, i, o, n, \quad i \le n \Longrightarrow s(i, n) < T(s)(o, n). \tag{3}$$

Notice that the output coordinate o is unrestricted. This asymmetry is useful: a visible seed coordinate can be placed below any selected output of a transformed family.

The repository already proves three generic facts.

1. A progressive transformer remains progressive when iterated down a W-tree by `runTransformer`.

2. Its autonomous lift along the universal W-language tree `index` remains progressive.

3. `operatorStage` recursively lifts a transformer-valued family along another W-tree and preserves progressivity.

These theorems are purely structural. They do not know whether the primitive reflection language is System T, System F, Fω, kind-polymorphic Fω, or the new Fd language.

3.3 Why oracle monotonicity is deliberately absent

One might hope to prove $s \le s'$ pointwise and infer $T(s) \le T(s')$. That is false for the reflected languages: object programs may inspect an oracle value and branch on it, so increasing an oracle can decrease a program's output. BusyLean's generic proofs correctly avoid this trap. Every strict inequality follows a concrete occurrence of a child or seed in a finite maximum, or a concrete path through a W-tree. The new proof follows the same discipline.

4 The new family-aware Fd reflection

Construction 4.1 (Family-aware dynamic-kind reflection)**.** For a family s and budget n, define

$$R_{14}(s,n) = n + 1 + \max\left(\max_{i \le n} s(i,n),\ R^{\mathsf{Fd}}_{\widehat{s}}(n)\right), \tag{4}$$

where $\widehat{s} = \mathrm{packFamily}(s)$ is its unary code.

The Lean definition is intentionally direct:

```
def reflect14 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 +
    BigNumber.maxN
      (W.prefixMax (fun i => fs i n) n)
      (Fd.reflectFd (packFamily fs) n)

def fdTransformer : Transformer := fun fs _ n => reflect14 fs n
```

All maxima in (4) are over finite lists, and the call to R^{Fd} is total by the incumbent development. Therefore R_{14} is a computable function in the ordinary operational sense, not merely a classically defined natural number.

Lemma 4.2 (Visible-family margin)**.** *For every $i \le n$,*

$$s(i,n) < R_{14}(s,n).$$

Proof. The value $s(i,n)$ is at most the prefix maximum, which is at most the outer maximum. The summand $n+1$ makes the inequality strict. The Lean theorem is `oracle_lt_reflect14`. □

Lemma 4.3 (Fd-program margin)**.** *If $t \in \mathcal{T}_{\mathsf{Fd}}(n)$, then*

$$\mathrm{value}_{\widehat{s}}(n,t) < R_{14}(s,n).$$

Proof. The existing theorem puts the program value below $R^{\mathsf{Fd}}_{\widehat{s}}(n)$; that unary reflection is explicitly below the outer maximum in (4). The Lean theorem is `fvalue_lt_reflect14`. □

Proposition 4.4 (Progressivity)**.** *The transformer `fdTransformer` is progressive.*

Proof. The output coordinate is ignored, and theorem 4.2 is exactly the predicate in (3). This is `fdTransformer_progressive`. □

5 A stronger autonomous schedule

Contender 12 introduced a W-tree schedule obtained from transparent ordinal-like syntax. The syntax has constructors for zero, successor, and countable limit, plus structural operations resembling addition, multiplication, omega powers, derivatives, a common-critical closure, and higher derivatives. No quotient by ordinal equality is required; only the generated tree matters.

Let T_{12} denote the existing `tree12`. The new fixed schedule is

$$T_{14} = \mathrm{realize}\,(\mathrm{hyper}_{42}(\mathrm{encode}(T_{12}))) \,. \tag{5}$$

In Lean:

```
def tree14 : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper 42 (Autonomous.encode tree12))
```

This is a second autonomous 42-level closure placed *above the whole schedule* used by Contender 12. The phrase "above" here concerns the syntactic embedding and the explicit child structure. The formal comparison with Contender 13 does not depend on assigning a quotient-level ordinal to T_{14}, so no delicate ordinal-identity claim enters the trusted argument.

6 Finite powers of complete operator stages

Fix a W-tree a and a base transformer B. Define transformer powers by

$$P_0^{a,B} = B, \qquad P_{k+1}^{a,B} = \mathsf{operatorStage}(P_k^{a,B}, a). \tag{6}$$

The executable Lean definition is only four lines:

```
def operatorPower (tree : WTree Nat) (base : Transformer) :
    Nat -> Transformer
  | 0 => base
  | k + 1 => operatorStage (operatorPower tree base k) tree
```

This is a finite recursion on k. Even when k is astronomically large, the function is computable in principle. The contest does not require feasible numerical evaluation; it requires a terminating definition and a feasible *type-checking* proof.

Theorem 6.1 (Preservation of progressivity)**.** *If B is progressive, then every $P_k^{a,B}$ is progressive.*

Proof. Induct on k. The zero case is the hypothesis. The successor case is exactly the repository theorem `operatorStage_progressive`. The formal theorem is `operatorPower_progressive`. □

The next lemma records a stronger structural margin that is useful for program domination.

Lemma 6.2 (Base below one complete stage)**.** *Let B be progressive. For every W-tree a, family s, and $i \le n$,*

$$B(s)(i,n) < \mathsf{operatorStage}(B,a)(s)(o,n)$$

for every output coordinate o.

Proof. At a leaf, `operatorStage` is the autonomous lift, and the existing theorem `transformer_lt_lift_at` gives the claim. At a node, recurse into child zero and then use `operatorStage_child_lt` to enter the parent. This follows an explicit path through the tree and assumes no oracle monotonicity. The Lean theorem is `transformer_lt_operatorStage`. □

Corollary 6.3 (Strict power chain)**.** *For visible input coordinates,*

$$P_k^{a,B}(s)(i,n) < P_{k+1}^{a,B}(s)(o,n).$$

Moreover, B lies below every positive power.

Proof. Apply theorem 6.2 first to P_k, then induct on the number of powers. The formal theorems are `operatorPower_step_lt` and `transformer_lt_operatorPower`. □

7 The self-height diagonal

Define the complete incumbent family

$$G(n) = F_{13}(F_{13}(n)). \tag{7}$$

Thus $G(42)$ is definitionally C_{13}, but the new source never mentions the constant `contender_13`. Let $\underline{G}$ be the constant-coordinate family $\underline{G}(i,n) = G(n)$. Use $B = \texttt{fdTransformer}$ and $a = T_{14}$ in (6).

Definition 7.1 (Contender 14 family)**.**

$$F_{14}(n) = P_{G(n)+1}^{T_{14},B}(\underline{G})(G(n), G(n)). \tag{8}$$

The proposed closed number is

$$C_{14} = F_{14}(F_{14}(42)). \tag{9}$$

The three appearances of $G(n)$ in (8) have distinct roles:

1. $G(n)+1$ is the number of *complete operator-stage rounds*;
2. the family argument supplies G as the oracle seed at every coordinate;
3. $G(n)$ is both the selected output coordinate and the numerical budget.

The Lean code is correspondingly explicit:

```
def family13 (n : Nat) : Nat := F13 (F13 n)

def F14 (n : Nat) : Nat :=
  operatorPower tree14 fdTransformer (family13 n + 1)
    (fun _ => family13) (family13 n) (family13 n)

def contender_14 : Nat := F14 (F14 42)
```

7.1 The strict comparison

Lemma 7.2 (The incumbent family exceeds its input)**.** *For all n, $n < G(n)$.*

Proof. The existing Fd input-margin theorem gives $n < F_{13}(n) < F_{13}(F_{13}(n))$. The Lean theorem is `input_lt_family13`. □

Lemma 7.3 (Self-height margin)**.** *For all n,*

$$G(G(n)) < F_{14}(n).$$

Proof. By theorem 6.1, the final transformer in (8) is progressive. Apply progressivity to seed coordinate zero, output coordinate $G(n)$, and numerical input $G(n)$. The seed value is $\underline{G}(0, G(n)) = G(G(n))$. This is the formal theorem `family13_twice_lt_F14`. □

Theorem 7.4 (Pointwise domination)**.** *For every n,*

$$G(n) < F_{14}(n).$$

Proof. By theorem 7.2 at input $G(n)$, $G(n) < G(G(n))$. Compose with theorem 7.3. This is `family13_lt_F14`. □

Theorem 7.5 (Required contest comparison)**.**

$$C_{13} < C_{14}.$$

Proof. By theorem 7.4 at 42 and the input margin for F_{14},

$$C_{13} = G(42) < F_{14}(42) < F_{14}(F_{14}(42)) = C_{14}.$$

The Lean theorem is `contender_13_lt_contender_14`. □

The comparison is therefore not a fragile numerical estimate. It is a short composition of strict inequalities generated by visible occurrences in the construction.

8 Program-level domination

The construction retains a semantic margin beyond the predecessor constant. Let t be any accepted dynamic-kind term at budget $G(n)$. By theorem 4.3,

$$\mathrm{value}_{\widehat{\underline{G}}}(G(n), t) < R_{14}(\underline{G}, G(n)).$$

By theorem 6.3, the base reflection output lies below the positive operator power in (8). Hence:

Theorem 8.1 (Diagonal Fd-program domination)**.** *If $t \in \mathcal{T}_{\mathsf{Fd}}(G(n))$, then*

$$\mathrm{value}_{\widehat{\underline{G}}}(G(n), t) < F_{14}(n).$$

The Lean theorem is `fvalue_lt_F14`. This matters conceptually. The new number does not merely iterate the old numerical family; its first reflection layer already majorizes the entire bounded Fd programming language relative to a packed family oracle, and the result is then passed through a self-height tower of complete operator closures.

9 Constructivity and contest compliance

Requirement	How the proposal satisfies it
Closed natural number	`contender_14` has type `Nat` and no free parameters.
Computable in principle	All recursions are structural: Fd normalization is already certified; maxima are finite; W-tree traversals use inductive recursors; `operatorPower` is primitive recursion on a natural number.

Requirement	How the proposal satisfies it
No previous constant	The definition uses `family13 n := F13 (F13 n)` and never references `contender_13` or any earlier closed contender.
Qualitative strengthening	A family-aware Fd reflection is lifted to a transformer, then to an operator stage, then iterated for a previous-family-sized height along a schedule above `tree12`.
Formal strict comparison	`contender_13_lt_contender_14` follows from the pointwise theorem `family13_lt_F14`.
Axiom discipline	The audit file prints the axiom closure of the number and key theorems and rejects dependencies on earlier contender constants.
No forbidden implementation device	The static audit rejects `noncomputable`, `partial`, `unsafe`, `opaque`, external implementations, `native_decide`, `sorry`, custom axioms, and `Classical.choice`.
No giant-term normalization in proofs	Proofs use named margin lemmas and explicit paths. They avoid simplification tactics that could unfold a contender-sized recursion.
Time budgets	The supplied clean-build harness separately times the previous accepted stack, the new definition module, and the new comparison module under the repository's 15-second and 60-second limits.

10 Heuristic growth analysis

This section is explanatory and is *not* part of the formal theorem. Exact proof-theoretic calibration of the custom Fd language and of the generic operator construction would require a separate ordinal analysis.

Let $\Lambda_{\mathtt{Fd}}$ denote the effective growth strength of one dynamic-kind reflection step, and let α_{12} and α_{14} denote heuristic heights associated with `tree12` and `tree14`. The source structure suggests

$$\alpha_{14} \approx H_{42}(\alpha_{12}),$$

where H_{42} stands only for the 42-level autonomous-derivative syntax used by the implementation. It is not asserted to be a canonical ordinal function.

A single Fd reflection, as in Contender 13, adds one bounded-language majorization at a very large diagonal input. A complete operator stage does more: it turns a transformer into a hierarchy of child transformers, takes an autonomous lift along the W-language's universal index at each node, and then iterates this operation through the external schedule. If one writes Φ_α schematically for the resulting closure on growth indices, the new family has the rough form

$$F_{14}(n) \text{ behaves like } \Phi_{\alpha_{14}}^{G(n)+1}(\text{Fd-reflection over } G)(G(n)),$$

where the superscript denotes finite iteration. Since the iteration count is itself $G(n)$, this is a self-height closure, not a fixed finite stack.

Three independent effects separate the proposal from earlier contenders:

1. the primitive language is Fd, stronger than the Fk language amplified by Contenders 11–12;
2. the schedule places an additional autonomous closure above Contender 12's entire schedule; and
3. the number of complete operator stages is not 42 or another fixed numeral but the full predecessor-family value $G(n) + 1$.

The only size claim needed for acceptance remains the exact Lean inequality in theorem 7.5. The heuristic picture explains why the margin should be regarded as substantial rather than cosmetic.

11 Lean proof engineering

11.1 Never normalize a contender

The largest practical danger is not logical but elaborational. A seemingly innocent `simp [F14]` or an aggressive `change` can ask Lean to compare terms by reducing a recursion whose index is $G(n)$. That computation is impossible in practice even though it terminates mathematically. The proof therefore unfolds only small outer wrappers and applies general lemmas whose parameters remain symbolic.

11.2 Use explicit lower-bound occurrences

Every margin theorem follows one of four patterns:

1. membership in a finite maximum;
2. a visible family coordinate in `prefixMax`;
3. a child-to-parent path in a W-tree; or
4. transitivity of already strict inequalities.

This makes the proof robust against the nonmonotonic behavior of programs with oracle access.

11.3 Separate definition and proof budgets

`Contender14.lean` contains only the seven executable definitions. The more elaborate structural lemmas live in `Comparison14.lean`. The check script first builds the accepted Contender 13 stack, then times the new definition module against 15 seconds and the proof module against 60 seconds. The source itself remains small: the definition module is under one hundred lines, and the proof module is under two hundred lines.

12 Validation status and reproducibility

The package was developed against the exact public signatures in the stated BusyLean snapshot. A source-level audit was run successfully: it found all required definitions and theorems, found no forbidden construct, found no reference to any earlier `contender_N` constant in the comment-stripped definition, and verified a 100-column source limit.

Validation note 12.1. The execution container used to prepare this archive did not contain the repository-pinned Lean 4.34.0 toolchain and could not retrieve its large binary archive through the available restricted network path. Consequently, this package does *not* claim that the two new Lean modules were kernel-built in that container. The archive includes `scripts/check14.py` and `test/Audit14.lean` so that compilation, timing, axiom inspection, and transitive dependency checks can be performed exactly in a BusyLean checkout.

The distinction is important. The mathematical argument and Lean source have been made as explicit and conservative as possible, but only a successful kernel run upgrades the proposal from a carefully audited formalization to a verified repository contender.

12.1 Reproduction commands

After copying the overlay files into a checkout at the target snapshot or a compatible successor:

```
cp BusyLean/Contender14.lean /path/to/BusyLean/BusyLean/
cp BusyLean/Comparison14.lean /path/to/BusyLean/BusyLean/
cp test/Audit14.lean /path/to/BusyLean/test/
cp scripts/check14.py /path/to/BusyLean/scripts/
cd /path/to/BusyLean
python3 scripts/check14.py
```

The integration patches additionally add the imports to `BusyLean.lean` and extend the repository-wide audit.

13 Further research

The present proposal maximizes leverage from already proved infrastructure. Several deeper successors are natural.

A genuine ladder of dynamic kinds. Introduce kinds $\mathtt{dyn}_0, \mathtt{dyn}_1, \ldots$, where $\mathtt{dyn}_{j+1}$ packages semantic operators built from lower levels. A budget-n language could expose the first n levels. The main challenge is a uniform candidates model and a fuel-bounded checker whose soundness proof does not force large computations.

Fd-definable schedules. The current schedule is built in Lean from the W-language's universal tree. A stronger object language with safe large elimination could return tree codes to Lean, allowing the dynamic-kind language itself to generate the reflection schedule.

Transfinite operator powers. Replace the finite `operatorPower` index by a W-tree recursion whose nodes carry whole progressive transformers. This must preserve computability and must expose enough seed occurrences to prove strictness without assuming monotonicity.

Exact ordinal analysis. Develop a conservative proof-theoretic calibration for dynamic-kind Fω, its family reflector, and the operator-stage closure. The article's current size discussion intentionally stops at structural heuristics.

Machine-checked metatheory of the contest rules. Formalize the source-closure audit, forbidden-construct policy, and timing interface as a reusable Lean or external checker with a small verified core. This would make future contender acceptance more reproducible.

A Complete definition module

```
import BusyLean.MetaReflection
import BusyLean.Autonomous
import BusyLean.Contender13

/-!
# Contender 14: self-height operator power over dynamic-kind System F-omega

Contender 13 performs one reflection over dynamic-kind System F-omega.  This
```

```
successor first turns that reflection into a progressive transformer on
countable oracle families, then applies the repository's whole operator-stage
construction repeatedly.

The fixed schedule `tree14` places another 42-level autonomous closure above
Contender 12's schedule.  At input `n`, the number of operator-stage rounds is
`family13 n + 1`, and the final transformer is evaluated at `family13 n`.
Thus the previous construction controls both the seed and the height of a
qualitatively stronger construction, without mentioning the constant for
Contender 13.

All definitions are structural and executable.  Do not evaluate
`contender_14`.
-/

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- Dynamic-kind reflection lifted from one unary oracle to a countable oracle
family.  The packed oracle is seen by object-language programs; the prefix
maximum makes every visible family coordinate an explicit lower bound. -/
def reflect14 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 +
    BigNumber.maxN
      (W.prefixMax (fun i => fs i n) n)
      (Fd.reflectFd (packFamily fs) n)

/-- One family-aware dynamic-kind reflection step, as a transformer. -/
def fdTransformer : Transformer := fun fs _ n => reflect14 fs n

/-- A schedule strictly more elaborate than Contender 12's schedule: another
42-level autonomous closure is stacked above `tree12`.  This is syntax for a
well-founded tree, not a claim about quotient-level ordinal identities. -/
def tree14 : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper 42 (Autonomous.encode tree12))

/-- Iterate the complete operator-stage construction along a fixed schedule.
Each successor level treats the preceding transformer as its new base step. -/
def operatorPower (tree : WTree Nat) (base : Transformer) :
    Nat → Transformer
  | 0 => base
  | k + 1 => operatorStage (operatorPower tree base k) tree

/-- Contender 13's construction as a function of its input.  In particular,
`family13 42` is definitionally the previous contender, but that constant is
not used here. -/
def family13 (n : Nat) : Nat := F13 (F13 n)

/-- The self-height dynamic-kind operator power.

At input `n`, perform `family13 n + 1` complete operator stages, seed every
oracle coordinate with `family13`, and evaluate at the previous-family value
`family13 n`. -/
def F14 (n : Nat) : Nat :=
  operatorPower tree14 fdTransformer (family13 n + 1)
    (fun _ => family13) (family13 n) (family13 n)
```

```
/-- **Contender 14.** Never evaluate it. -/
def contender_14 : Nat := F14 (F14 42)

end BusyLean
```

B Complete comparison module

```
import BusyLean.Contender14
import BusyLean.Comparison13
import BusyLean.MetaReflectionFacts

/-! # Contender 14: comparison and structural-margin theorems -/

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-! ## The family-aware dynamic-kind reflection step -/

/-- Anything bounded by the explicit maximum is strictly below `reflect14`. -/
theorem lt_reflect14 {fs : W.Family} {n m : Nat}
    (h : m ≤ BigNumber.maxN
      (W.prefixMax (fun i => fs i n) n)
      (Fd.reflectFd (packFamily fs) n)) :
    m < reflect14 fs n := by
  show m < n + 1 + _
  have hM : ∀ M : Nat, M + 1 ≤ n + 1 + M := fun M => by
    rw [Nat.add_comm M 1]
    exact Nat.add_le_add_right (Nat.le_add_left 1 n) M
  exact Nat.lt_of_le_of_lt h
    (Nat.lt_of_lt_of_le (Nat.lt_succ_self _) (hM _))

/-- Every family coordinate visible at the current input is strictly below the
new reflection step. -/
theorem oracle_lt_reflect14 (fs : W.Family) {i n : Nat}
    (hi : i ≤ n) : fs i n < reflect14 fs n :=
  lt_reflect14
    (Nat.le_trans
      (W.le_prefixMax (fun j => fs j n) hi)
      (BigNumber.le_maxN_left _ _))

/-- The underlying unary dynamic-kind reflection is itself strictly below the
family-aware reflection step. -/
theorem packed_reflection_lt_reflect14 (fs : W.Family) (n : Nat) :
    Fd.reflectFd (packFamily fs) n < reflect14 fs n :=
  lt_reflect14 (BigNumber.le_maxN_right _ _)

/-- Every accepted dynamic-kind program at the current budget is strictly
below the family-aware reflection step. -/
theorem fvalue_lt_reflect14 (fs : W.Family) {n : Nat} {t : Fd.CTm}
    (ht : t ∈ Fd.ctmsUpTo n) :
    Fd.fvalue (packFamily fs) n t < reflect14 fs n :=
  Nat.lt_trans
    (Fd.fvalue_lt_reflectFd (packFamily fs) ht)
    (packed_reflection_lt_reflect14 fs n)
```

```
/-- The numerical input is strictly below every family-aware reflection step. -/
theorem input_lt_reflect14 (fs : W.Family) (n : Nat) :
    n < reflect14 fs n :=
  Nat.lt_of_lt_of_le (Nat.lt_succ_self n) (Nat.le_add_right _ _)

/-- The base dynamic-kind transformer is progressive. -/
theorem fdTransformer_progressive : Progressive fdTransformer := by
  intro seed i o n hi
  exact oracle_lt_reflect14 seed hi

/-! ## Generic operator-power lemmas -/

/-- Progressivity is preserved by every finite operator power. -/
theorem operatorPower_progressive (tree : WTree Nat)
    {base : Transformer} (hbase : Progressive base) :
    ∀ k, Progressive (operatorPower tree base k)
  | 0 => hbase
  | k + 1 =>
      operatorStage_progressive
        (operatorPower tree base k)
        (operatorPower_progressive tree hbase k)
        tree

/-- A progressive transformer is strictly below one complete operator stage,
at every visible input coordinate.  The proof follows coordinate zero through
one child at every node; no monotonicity in the oracle is assumed. -/
theorem transformer_lt_operatorStage (base : Transformer)
    (hbase : Progressive base) (seed : Family) :
    ∀ (a : WTree Nat) (i o n : Nat), i ≤ n →
      base seed i n < operatorStage base a seed o n
  | .leaf, i, o, n, hi =>
      transformer_lt_lift_at (op := base) hbase seed
        (i := i) (o := o) (n := n) hi
  | .node f, i, o, n, hi =>
      Nat.lt_trans
        (transformer_lt_operatorStage base hbase seed
          (f 0) i 0 n hi)
        (operatorStage_child_lt base hbase seed f
          (i := 0) (o := o) (n := n) (Nat.zero_le n))

/-- Consecutive operator powers form a strict chain. -/
theorem operatorPower_step_lt (tree : WTree Nat)
    (base : Transformer) (hbase : Progressive base) (seed : Family)
    (k i o n : Nat) (hi : i ≤ n) :
    operatorPower tree base k seed i n <
      operatorPower tree base (k + 1) seed o n :=
  transformer_lt_operatorStage
    (operatorPower tree base k)
    (operatorPower_progressive tree hbase k)
    seed tree i o n hi

/-- Every positive operator power strictly exceeds the original base
transformer. -/
theorem transformer_lt_operatorPower (tree : WTree Nat)
    (base : Transformer) (hbase : Progressive base) (seed : Family) :
    ∀ (k i o n : Nat), i ≤ n →
      base seed i n < operatorPower tree base (k + 1) seed o n
  | 0, i, o, n, hi =>
      transformer_lt_operatorStage base hbase seed tree i o n hi
  | k + 1, i, o, n, hi =>
```

```
      Nat.lt_trans
        (transformer_lt_operatorPower tree base hbase seed
          k i 0 n hi)
        (transformer_lt_operatorStage
          (operatorPower tree base (k + 1))
          (operatorPower_progressive tree hbase (k + 1))
          seed tree 0 o n (Nat.zero_le n))

/-! ## The diagonal family and the required comparison -/

/-- Contender 13's family strictly exceeds its input. -/
theorem input_lt_family13 (n : Nat) : n < family13 n :=
  Nat.lt_trans
    (Fd.input_lt_reflectFd family12 n)
    (Fd.input_lt_reflectFd family12 (F13 n))

/-- The previous family, evaluated once more at its own value, is still below
`F14`.  This is the direct progressivity margin of the final transformer. -/
theorem family13_twice_lt_F14 (n : Nat) :
    family13 (family13 n) < F14 n := by
  change family13 (family13 n) <
    operatorPower tree14 fdTransformer (family13 n + 1)
      (fun _ => family13) (family13 n) (family13 n)
  exact
    (operatorPower_progressive tree14 fdTransformer_progressive
      (family13 n + 1))
      (fun _ => family13) 0 (family13 n) (family13 n)
      (Nat.zero_le (family13 n))

/-- The base family-aware reflection step at the diagonal budget lies below the
whole positive operator power. -/
theorem reflect14_lt_F14 (n : Nat) :
    reflect14 (fun _ => family13) (family13 n) < F14 n := by
  change fdTransformer (fun _ => family13) 0 (family13 n) <
    operatorPower tree14 fdTransformer (family13 n + 1)
      (fun _ => family13) (family13 n) (family13 n)
  exact transformer_lt_operatorPower
    tree14 fdTransformer fdTransformer_progressive
    (fun _ => family13) (family13 n) 0
    (family13 n) (family13 n) (Nat.zero_le (family13 n))

/-- At every input, `F14` strictly exceeds Contender 13's complete family. -/
theorem family13_lt_F14 (n : Nat) : family13 n < F14 n :=
  Nat.lt_trans
    (input_lt_family13 (family13 n))
    (family13_twice_lt_F14 n)

/-- The final diagonal strictly exceeds its own numerical input. -/
theorem input_lt_F14 (n : Nat) : n < F14 n :=
  Nat.lt_trans (input_lt_family13 n) (family13_lt_F14 n)

/-- Every accepted dynamic-kind program at the diagonal budget, using the
packed constant family `family13`, is below `F14 n`. -/
theorem fvalue_lt_F14 {n : Nat} {t : Fd.CTm}
    (ht : t ∈ Fd.ctmsUpTo (family13 n)) :
    Fd.fvalue (packFamily (fun _ => family13)) (family13 n) t < F14 n :=
  Nat.lt_trans
    (fvalue_lt_reflect14 (fun _ => family13) ht)
    (reflect14_lt_F14 n)
```

```
/-- **The required comparison:** Contender 13 is strictly smaller than
Contender 14. -/
theorem contender_13_lt_contender_14 : contender_13 < contender_14 := by
  change family13 42 < F14 (F14 42)
  exact Nat.lt_trans (family13_lt_F14 42) (input_lt_F14 (F14 42))

/-- All earlier accepted contenders are therefore also below Contender 14. -/
theorem contender_12_lt_contender_14 : contender_12 < contender_14 :=
  Nat.lt_trans contender_12_lt_contender_13 contender_13_lt_contender_14

end BusyLean
```

C Theorem inventory

Lean theorem	Role
`lt_reflect14`	Generic bound-to-strict-margin lemma.
`oracle_lt_reflect14`	Every visible family coordinate is dominated.
`packed_reflection_lt_reflect14`	The embedded unary Fd reflection is dominated.
`fvalue_lt_reflect14`	Every accepted Fd program at the current budget is dominated.
`fdTransformer_progressive`	Discharges the generic operator machinery's hypothesis.
`operatorPower_progressive`	Every finite complete-stage power is progressive.
`transformer_lt_operatorStage`	Explicit path from a base transformer into one complete stage.
`operatorPower_step_lt`	Consecutive powers form a strict chain.
`transformer_lt_operatorPower`	The base lies below every positive power.
`input_lt_family13`	The incumbent family exceeds its input.
`family13_twice_lt_F14`	Direct self-height seed margin.
`reflect14_lt_F14`	Base family reflection lies below the whole power.
`family13_lt_F14`	Pointwise domination of the incumbent family.
`input_lt_F14`	New family exceeds its input.
`fvalue_lt_F14`	Program-level domination at the diagonal budget.
`contender_13_lt_contender_14`	Required contest theorem.

References

[Aar99] Scott Aaronson. *Who Can Name the Bigger Number?* Web essay, 1999. https://www.scottaaronson.com/writings/bignumbers.html.

[BR26] Vladimir Reshetnikov and contributors. *BusyLean: Name the Biggest Number, in Lean.* GitHub repository, 2026; snapshot `0cd13c0c2607b4fcb461090a95f356e538c0fc5b`. https://github.com/VladimirReshetnikov/BusyLean.

[DMU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, LNCS 12699, pages 625–635. Springer, 2021.

[GLT89] Jean-Yves Girard, Yves Lafont, and Paul Taylor. *Proofs and Types.* Cambridge University Press, 1989.

[Rey74] John C. Reynolds. Towards a theory of type structure. In *Programming Symposium*, LNCS 19, pages 408–425, 1974.

[Rou14] Cody Roux and contributors. *Name the Biggest Number.* GitHub repository, 2014. https://github.com/codyroux/name-the-biggest-number.